Universidad del Valle de Guatemala
Facultad de Ingeniería
Departamento de Ciencias de la Computación
CC3104 Aprendizaje por Refuerzo

RiegoRL: aprendizaje por refuerzo para el riego de maíz en el Corredor Seco de Guatemala

Proyecto final

José Luis Gramajo Moraga, carné 22907
Ángel Andrés Herrarte Lorenzana, carné 22873

Repositorio: https://github.com/Abysswalkr/RiegoRL-FinalP_AR
Video: https://youtu.be/rxHjj0YbTBg

Guatemala, 9 de noviembre de 2026

Índice

Resumen **3**

1. Descripción del problema **4**

2. Análisis **5**

2.1 Modelo del sistema suelo-cultivo 5

2.2 Formulación como proceso de decisión de Markov 6

2.3 Datos y división en conjuntos 6

2.4 Estrategias de referencia 6

3. Propuesta de solución **7**

4. Descripción de la solución **8**

4.1 Organización del software 8

4.2 Agente Q-learning 8

4.3 Agente DQN 9

4.4 Proceso iterativo 9

4.5 Configuración final y reproducibilidad 11

5. Herramientas aplicadas **12**

5.1 Herramientas vistas en clase 12

5.2 Herramientas no vistas en clase 12

5.3 Uso de inteligencia artificial generativa 13

6. Resultados **16**

7. Conclusiones **20**

7.1 Limitaciones y trabajo futuro 21

8. Bibliografía **21**

Resumen

Este proyecto trata una decisión que los productores del Corredor Seco de Guatemala toman a diario: cuánta agua aplicar a una parcela de maíz. Construimos un entorno de aprendizaje por refuerzo que simula el balance hídrico del suelo con el método FAO-56 y 26 años de clima de Chiquimula, y sobre él implementamos dos agentes desde cero: Q-learning tabular y una red Q profunda (DQN). Ambos se compararon con tres estrategias de referencia (no regar, regar por calendario y regar cuando la humedad del suelo cruza un umbral) en dos escenarios, uno con agua ilimitada y otro con un presupuesto de 150 mm por temporada.

En la evaluación final, sobre las temporadas de 2022 a 2025, DQN obtuvo la misma recompensa que la regla por umbral cuando no hay presupuesto (131.4 contra 131.3) y tuvo menos días de estrés. Con presupuesto, la regla por umbral fue la mejor estrategia. La causa está en el simulador: su modelo de rendimiento valora igual el agua en todas las etapas del cultivo, así que reservarla para la floración no mejora el resultado.

1. Descripción del problema

El Corredor Seco Centroamericano es una franja de clima semiárido con lluvias escasas e irregulares. En Guatemala ocupa buena parte del oriente del país, incluido el departamento de Chiquimula, donde la mayoría de productores siembra maíz y frijol para autoconsumo. Las cosechas dependen de la lluvia y se pierden con frecuencia por sequías o por la canícula, el periodo seco que interrumpe la temporada de lluvias a mediados del año (FAO, 2012). Cuando hay posibilidad de regar, el agua es limitada y cuesta dinero, y el productor tiene que decidir cada día si riega y cuánto.

En los datos que usamos, Chiquimula recibe en promedio 1 062 mm de lluvia al año, mientras la evapotranspiración de referencia (ET_0) suma 1 457 mm. La lluvia se concentra en dos temporadas de siembra. Durante el ciclo de la *primera* (siembra el 15 de mayo) caen en promedio 677 mm, y durante el de la *postrera* (1 de septiembre), 414 mm. En la *época seca* (15 de noviembre) caen 77 mm en promedio y hay años con apenas 28 mm. Sin riego, el maíz sembrado en época seca pierde en nuestros datos más del 70 % de su rendimiento.

Decidir el riego es un problema secuencial con incertidumbre. Regar hoy reduce el estrés de la planta, pero si mañana llueve, parte del agua se pierde por percolación. No regar ahorra agua, pero si la sequía se alarga, la planta transpira menos y el rendimiento baja. El rendimiento solo se conoce al final de la temporada, después de 125 decisiones diarias. El aprendizaje por refuerzo está pensado para problemas así, en los que una secuencia de decisiones produce una recompensa que llega tarde.

El objetivo del proyecto es entrenar un agente que decida cada día cuánta agua aplicar (0, 10, 20, 30 o 40 mm) para obtener el mayor rendimiento relativo posible con poca agua y pocos días de estrés hídrico. El agente se compara con estrategias de referencia en dos escenarios: con agua ilimitada y con un presupuesto de 150 mm por temporada.

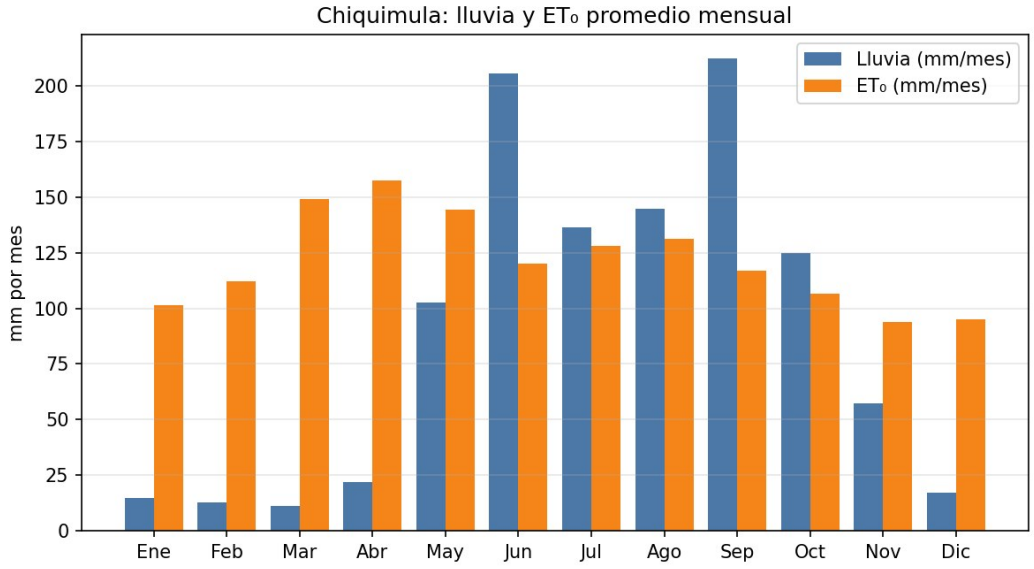


Figura 1. Lluvia y ET_0 promedio por mes en Chiquimula, 2000-2026. Entre noviembre y abril la demanda de agua supera con mucho a la lluvia.

2. Análisis

2.1 Modelo del sistema suelo-cultivo

Para entrenar un agente hace falta un entorno que responda a sus decisiones. Usamos el balance hídrico de una capa de suelo descrito en el manual FAO-56 (Allen et al., 1998), el método con el que normalmente se programa el riego. La variable central es la depleción D_r, es decir, los milímetros de agua que le faltan al suelo para llegar a capacidad de campo. Cada día se actualiza así:

$$D_{r,i} = D_{r,i-1} - (P_i - RO_i) - I_i + ET_{a,i} + DP_i$$

donde P es la lluvia, RO la escorrentía, I el riego, ET_a la evapotranspiración real y DP la percolación profunda, el agua que sobra cuando el suelo se llena. El agua que la raíz puede aprovechar es $TAW = 1000\ (\theta_{FC} - \theta_{WP})\ Z_r$, y la parte que extrae sin esfuerzo es $RAW = p \cdot TAW$. Mientras $D_r \leq RAW$, la planta transpira a su ritmo máximo, $ET_c = K_c \cdot ET_0$. Cuando la depleción pasa ese umbral, la transpiración se reduce con el coeficiente de estrés:

$$K_s = (TAW - D_r) / (TAW - RAW),\ ET_a = K_s \cdot ET_c$$

Al final de la temporada, el rendimiento relativo se calcula con la función de respuesta al agua de FAO-33 (Doorenbos y Kassam, 1979):

$$Y_a / Y_m = 1 - K_y\,(1 - \Sigma ET_a / \Sigma ET_c)$$

Los parámetros corresponden a maíz de grano en suelo franco: una temporada de 125 días con etapas de 20, 35, 40 y 30 días, K_c de 0.30, 1.20 y 0.35, raíz de 0.15 a 1.00 m, $\theta_{FC} = 0.30$, $\theta_{WP} = 0.15$ y $K_y = 1.25$.

2.2 Formulación como proceso de decisión de Markov

Cada temporada de cultivo es un episodio de 125 pasos diarios. La Tabla 1 resume el MDP.

Tabla 1
Formulación del problema como MDP

Elemento	Definición
Estado observado	Seis valores normalizados en [0, 1]: depleción relativa D_r/TAW, avance de la temporada, etapa fenológica, pronóstico de lluvia a 3 días, ET_0 del día y fracción de agua restante del presupuesto (1.0 si no hay presupuesto).
Acciones	Cinco dosis de riego: 0, 10, 20, 30 o 40 mm.
Transición	Balance hídrico FAO-56 con la lluvia y la ET_0 reales del día. Si el presupuesto no alcanza, se aplica lo que queda.
Recompensa diaria	$ET_a/ET_c - 0.3 \cdot I/40 - 0.5 \cdot DP/40$. Premia la transpiración y castiga tanto el agua aplicada como la que se pierde por percolación.
Recompensa final	$10 \cdot Y_a/Y_m$, que se suma el último día.
Descuento	$\gamma = 0.99$. Con este valor, el bono final todavía pesa al inicio de la temporada ($\gamma^{124} \approx 0.29$).

El agente no observa el estado completo. No conoce la lluvia futura, solo un pronóstico sintético que toma la lluvia real de los próximos 3 días, le aplica ruido lognormal ($\sigma = 0.5$) y en el 15 % de los casos reporta cero aunque vaya a llover. Además, cada temporada empieza con una humedad inicial aleatoria.

2.3 Datos y división en conjuntos

El clima diario (lluvia, ET_0 y temperaturas) proviene del archivo histórico de Open-Meteo para el punto 14.80° N, 89.54° O, en Chiquimula. Son 9 678 días entre enero de 2000 y junio de 2026, sin datos faltantes. Cada año aporta tres temporadas, una por fecha de siembra. Para que el entrenamiento no use información de años posteriores, las temporadas se dividen por año: entrenamiento con 2000-2019 (60 temporadas), validación con 2020-2021 (6) y prueba con 2022-2025 (12).

2.4 Estrategias de referencia

Antes de entrenar agentes evaluamos tres estrategias simples. La primera no riega. La segunda, de calendario, aplica 30 mm cada 5 días. La tercera, de umbral, riega cuando D_r supera RAW con la dosis más cercana a la depleción. La Tabla 2 muestra sus resultados en validación.

Tabla 2
Estrategias de referencia en validación (6 temporadas, 3 semillas)

Estrategia	Agua (mm)	Días con estrés	Rend. relativo	Recompensa
Sin riego	0	38.9	0.708	106.4
Calendario	750	0.0	1.000	119.8
Umbral	140	3.5	0.997	131.2
Umbral, presupuesto de 150 mm	95	15.9	0.931	124.7

La regla por umbral logra el 99.7 % del rendimiento máximo con 140 mm, así que deja muy poco margen para un agente. Por eso agregamos un segundo escenario, más cercano a la situación del Corredor Seco: un presupuesto de 150 mm por temporada. Con ese límite la regla se queda sin agua en la época seca y su rendimiento baja a 0.931. Nuestra hipótesis era que un agente que observa el agua restante podría aprender a guardarla para la etapa más sensible del cultivo y superar a la regla.

3. Propuesta de solución

Para resolver el MDP elegimos dos métodos vistos en el curso y los implementamos desde cero, sin bibliotecas de aprendizaje por refuerzo, para poder explicar cada parte. El primero es Q-learning tabular (Watkins y Dayan, 1992) sobre una versión discretizada del estado; es el método de diferencia temporal más sencillo y su política se puede revisar celda por celda. El segundo es Deep Q-Network (Mnih et al., 2015), que aproxima la función Q con una red neuronal sobre la observación continua y usa un buffer de experiencia y una red objetivo.

La forma de evaluar se fijó antes de entrenar. Cada agente se entrena con las semillas 0, 1 y 2 en las 60 temporadas de entrenamiento. Los hiperparámetros y las iteraciones de diseño se deciden con el conjunto de validación, y el de prueba se usa una sola vez, al final, sin reentrenar. Las cinco estrategias se evalúan en las mismas temporadas, con las mismas semillas y en los dos escenarios.

Las métricas son el agua aplicada, los días con estrés ($K_s < 1$), el rendimiento relativo, la productividad del agua (rendimiento por cada 100 mm aplicados) y la recompensa total. Acordamos también que, si un agente no superaba a la regla por umbral, lo reportaríamos y buscaríamos la causa en lugar de ajustar la evaluación.

4. Descripción de la solución

4.1 Organización del software

El proyecto está escrito en Python como un paquete con pruebas automáticas. La Tabla 3 describe sus partes.

Tabla 3
Organización del repositorio

Archivo o carpeta	Contenido
`riego_rl/suelo.py`	Balance hídrico FAO-56 de una capa de suelo.
`riego_rl/entorno.py`	Entorno *RiegoEnv* con la interfaz de Gymnasium.
`riego_rl/baselines.py`	Estrategias sin riego, calendario y umbral.
`riego_rl/evaluacion.py`	Función común de evaluación de políticas.
`riego_rl/qlearning.py`	Discretización del estado y agente Q-learning.
`riego_rl/dqn.py`	Red Q, buffer de experiencia y entrenamiento de DQN.
`scripts/`	Entrenamiento, evaluación final, figuras, animación y un script que corre todo el proyecto desde cero.
`tests/`	42 pruebas con pytest, entre ellas la de la regla de actualización y la del cálculo del error TD.

4.2 Agente Q-learning

El estado se discretiza en la depleción relativa (10 intervalos) y la etapa fenológica (4). Con presupuesto se agrega el agua restante en 3 intervalos. La primera versión incluía también el pronóstico de lluvia en 3 intervalos (0-5, 5-20 y más de 20 mm), que se quitó en la iteración 3. La tabla se actualiza con la regla de Q-learning, que es una aproximación estocástica de la ecuación de optimalidad de Bellman:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

Es un método *off-policy*: el agente explora con una política ε-greedy, pero el objetivo usa el máximo sobre las acciones siguientes, que corresponde a la política greedy. ε baja linealmente de 1.0 a 0.05 durante el 70 % de 20 000 episodios. Cada 500 episodios se evalúa la política greedy en validación y se guarda la mejor tabla.

4.3 Agente DQN

La función Q se aproxima con un perceptrón multicapa $6 \rightarrow 64 \rightarrow 64 \rightarrow 5$ con activaciones ReLU, que recibe la observación continua y devuelve el valor de las cinco dosis. El entrenamiento usa los dos mecanismos con los que Mnih et al. (2015) estabilizaron DQN. El primero es un buffer de experiencia con las últimas 50 000 transiciones, del que cada paso de gradiente toma un lote aleatorio de 64. Esto rompe la correlación entre días consecutivos de una misma temporada y permite usar cada transición muchas veces. El segundo es una red objetivo, una copia de la red que se actualiza cada 1 000 pasos y con la que se calcula el valor al que se ajusta Q(s, a):

$$y = r + \gamma\,(1 - terminal)\,\max_{a'} Q_{objetivo}(s', a')$$

Si ese valor se calculara con la misma red que se está entrenando, cambiaría en cada paso de gradiente y el entrenamiento podría oscilar. La pérdida es Huber, que limita el efecto de errores grandes como el del bono final, y el optimizador es Adam (Kingma y Ba, 2015) con tasa de aprendizaje 10^{-3}. ε baja de 1.0 a 0.05 durante la primera mitad de 2 000 episodios (250 000 pasos) y se guarda la red con mejor recompensa en validación.

4.4 Proceso iterativo

El diseño final salió de cuatro iteraciones. Cada una partió de una hipótesis, hizo un cambio y se decidió con los resultados en validación. La Tabla 4 las resume y el detalle está en `docs/ITERACIONES.md`.

Tabla 4

Iteraciones de diseño

Iteración	Hipótesis y cambio	Resultado en validación	Decisión
Versión inicial	Q-learning con Q inicial en 0 y α = 0.1 constante; DQN según la especificación.	Q-learning riega 399 mm (umbral: 140) y obtiene 126.8 de recompensa. DQN empata con el umbral: 131.4 contra 131.2.	Diagnosticar Q-learning.
1. Exploración y ruido	El agente nunca exploró el suelo seco: la exploración uniforme aplica 20 mm/día y, con Q = 0, los estados no visitados parecían malos. Cambio: Q inicial optimista (80) y α decreciente de 0.1 a 0.01.	El agua baja de 399 a 216 mm y la recompensa sube de 126.8 a 129.8.	Se adopta.
2. Presupuesto de 150 mm	Con agua limitada, el agente podría guardarla para la etapa más sensible.	Ningún agente supera a la regla (DQN 124.1, Q-learning 119.2, umbral 124.7). Tampoco la superan variantes manuales de la regla con reservas, dosis fijas o umbrales más tardíos.	Se reporta. Con un K_y único el agua vale lo mismo en todas las etapas.
3. Quitar el pronóstico	El pronóstico tiene mucho ruido y triplica los estados de la tabla.	Q-learning sube a 130.8 sin presupuesto y a 121.6 con presupuesto. DQN no cambia (131.35).	Se adopta en Q-learning; DQN conserva el pronóstico.
4. λ_{agua} = 0.6	Castigar más el agua quitaría riegos innecesarios con presupuesto.	Mejoras de 0.2 puntos, muy por debajo de la variación entre episodios (unos 10).	Se descarta.

Las Figuras 2 y 3 muestran el efecto de las iteraciones en la política de Q-learning. En la versión inicial, las celdas con depleción mayor que 0.4 nunca se visitaron y la dosis elegida variaba sin patrón. En la versión final el agente no riega hasta que la depleción llega a 0.4-0.6, cerca de donde empieza el estrés (RAW $\approx$ 0.55 $\cdot$ TAW). Llegó por su cuenta a una política parecida a la regla por umbral.

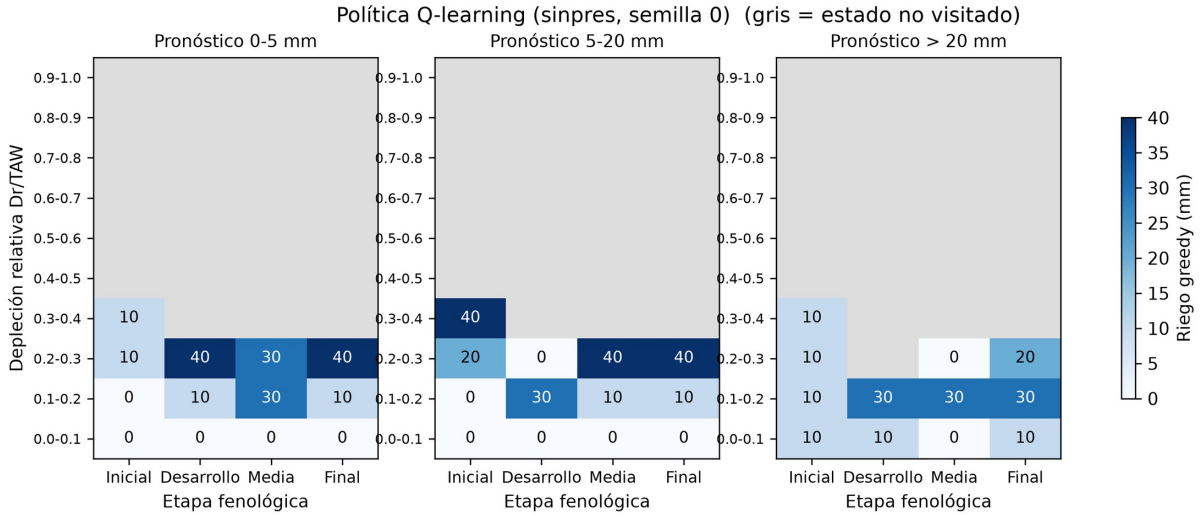


Figura 2. Política de la primera versión de Q-learning, sin presupuesto. Cada celda indica el riego greedy en mm; las grises son estados no visitados.

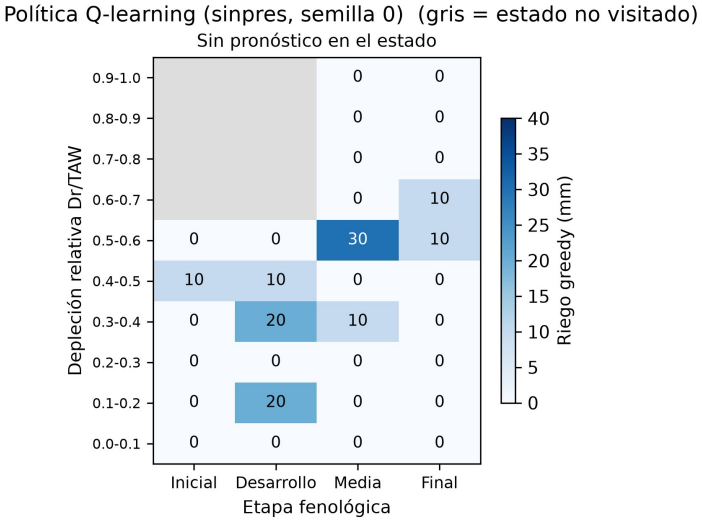


Figura 3. Política final de Q-learning, sin presupuesto y sin pronóstico. Las celdas muy secas de las etapas media y final solo se visitaron durante la exploración inicial y la política greedy no llega a ellas.

4.5 Configuración final y reproducibilidad

La configuración final está marcada en `config.py`. Todo el proyecto se reproduce con el comando `python scripts/correr_pipeline.py`, que corre las pruebas, entrena los 12 modelos finales (dos agentes, dos escenarios y tres semillas), evalúa y genera las figuras y la animación. La Tabla 5 muestra los tiempos en una computadora portátil de 10 núcleos, solo con CPU. Al correrlo desde cero, la tabla de resultados salió idéntica a la original.

Tabla 5
Tiempos del pipeline completo

Paso	Tiempo (s)
Pruebas	10
Baselines en val	2
Q-learning sin presupuesto	91
Q-learning presupuesto 150 mm	87
DQN sin presupuesto	141
DQN presupuesto 150 mm	207
Evaluación final en test	8
Figuras	11
Demo	69
Total	626

5. Herramientas aplicadas

5.1 Herramientas vistas en clase

Del curso usamos Q-learning y DQN con exploración ε-greedy (secciones 4.2 y 4.3). El entorno sigue la interfaz de Gymnasium (Towers et al., 2024): implementa los métodos *reset* y *step* y pasa el verificador *check_env* de la biblioteca. La red de DQN está escrita en PyTorch (Paszke et al., 2019). La tabla Q y el manejo de resultados usan NumPy y pandas, y las figuras, Matplotlib.

5.2 Herramientas no vistas en clase

Método FAO-56 de balance hídrico

Es el procedimiento de la Organización de las Naciones Unidas para la Alimentación y la Agricultura para estimar el consumo de agua de un cultivo y programar el riego (Allen et al., 1998). Parte de la evapotranspiración de referencia ET_0, que es la de un pasto bien regado, y la ajusta al cultivo con un coeficiente K_c. Ese coeficiente es bajo cuando la planta es pequeña, llega a su máximo en la floración y baja otra vez al madurar. El suelo se trata como un depósito cuya capacidad útil (TAW) crece con la raíz. Mientras el depósito conserva más agua que la fracción fácilmente aprovechable, la planta no sufre; por debajo de ese nivel, la transpiración cae en línea recta (sección 2.1). Lo elegimos porque es el método que usan los agrónomos para programar riego y porque cabe en pocas líneas de código: una temporada completa se simula en menos de 2 ms.

Respuesta del rendimiento al agua (FAO-33)

Relaciona la pérdida relativa de rendimiento con el déficit relativo de evapotranspiración mediante un factor K_y (Doorenbos y Kassam, 1979). Con K_y = 1.25, una caída del 10 % en la evapotranspiración produce una pérdida del 12.5 % en el rendimiento. Usamos un solo factor para toda la temporada, aunque FAO-33 propone uno por etapa. Esa simplificación terminó definiendo los resultados con presupuesto (sección 6).

Open-Meteo y el reanálisis ERA5

Open-Meteo es un servicio abierto de datos meteorológicos (Zippenfenig, 2023). Su archivo histórico ofrece series diarias desde 1940 calculadas a partir de reanálisis como ERA5 (Hersbach et al., 2020), que combina modelos numéricos del clima con observaciones. Gracias a él obtuvimos la lluvia y la ET_0 de Chiquimula sin depender de una estación meteorológica local.

pytest

Es el marco de pruebas automáticas más usado en Python. Lo usamos para verificar el simulador, el entorno, la evaluación y los agentes. Por ejemplo, una prueba compara la actualización de Q-learning con el cálculo hecho a mano y otra revisa que el objetivo de DQN no use el estado siguiente en el último día de la temporada.

5.3 Uso de inteligencia artificial generativa

Usamos Claude, en su versión de chat, para elegir el tema, formular el problema y analizar resultados, y Claude Code para escribir el código directamente en el repositorio (Anthropic, 2026). La idea cambió en tres momentos. Primero, de cuatro temas propuestos elegimos el riego por su aplicación en Guatemala. Después, al ver que la regla por umbral era casi óptima, agregamos el presupuesto de agua para darle margen al agente. Al final, cuando la hipótesis del presupuesto no se cumplió, buscamos la causa en lugar de cambiar el experimento hasta obtener un resultado favorable.

La Tabla 6 resume los prompts más relevantes y la Tabla 7, las respuestas que tuvieron errores. El registro completo está en `docs/REGISTRO_IA.md`.

Tabla 6

Historial de prompts relevantes

N.º	Herramienta	Prompt	Resultado	Evaluación
1	Claude (chat)	Con base en el enunciado, proponer temas de RL con aplicación real y una complejidad adecuada para un grupo de dos personas.	Comparó cuatro opciones según dificultad, riesgo de tema repetido y aplicación real, y recomendó el riego inteligente para cultivos.	Satisfactorio. Consideró las reglas del enunciado y descartó temas parecidos a trabajos previos del curso. La elección final la hicimos nosotros.
2	Claude (chat)	Formular el problema de riego como un MDP y planificar su implementación por fases para Claude Code.	Definió estado, acciones, transición y recompensa con base en el balance hídrico FAO-56 y clima real de Chiquimula. Generó un prompt por fases para implementar la base.	Parcial. La formulación fue la base de todo el proyecto, pero el prompt ordenaba hacer commits automáticos. Ver E1.
3	Claude Code	Implementar la base: descarga de datos climáticos, simulador de suelo, entorno Gymnasium, estrategias de referencia, evaluación y pruebas.	Datos de 9 678 días sin faltantes, simulador, entorno validado con check_env, tres estrategias de referencia y 28 pruebas que pasan.	Satisfactorio. Cumplió todas las fases y documentó sus decisiones en los puntos ambiguos, que revisamos y aceptamos. Ver E2, E3 y E4.
4	Claude (chat)	Analizar los primeros resultados de las estrategias de referencia y revisar el repositorio antes de continuar.	Detectó que la regla por umbral alcanzaba 0.997 de rendimiento con 140 mm, lo que dejaba casi sin margen al agente, y propuso un presupuesto de agua por temporada.	Satisfactorio. Fue el aporte más valioso. Cambió el diseño del problema hacia un escenario realista de racionamiento, donde el agente debe planificar el uso del agua.
5	Claude Code	Agregar el presupuesto de agua al entorno, simplificar los comentarios del código y comparar las estrategias con y sin presupuesto.	Presupuesto configurable, observación de 6 valores y 34 pruebas. Con 150 mm, la regla por umbral baja de 0.997 a 0.931 de rendimiento.	Satisfactorio. Verificó que sin presupuesto los resultados no cambiaran y separó los resultados de las hipótesis aún no probadas. Ver E5.
6	Claude Code	Implementar Q-learning tabular a mano con numpy, con estado discretizado, exploración ε-greedy y evaluación periódica en validación.	Código y pruebas correctos a la primera, pero la política aprendida regaba 399 mm, contra 140 mm de la regla por umbral.	Parcial. El código hacía lo pedido; el problema estaba en los hiperparámetros que especificamos. Se diagnosticó con el mapa de calor de la política antes de cambiar nada. Ver E6 y E9.
7	Claude Code	Implementar DQN a mano en PyTorch, con replay y red objetivo, y convertirlo en una política compatible con la evaluación.	Funcionó a la primera y empató con la regla por umbral sin presupuesto: 131.4 contra 131.2 de recompensa.	Satisfactorio. Se verificó la pérdida TD con una prueba que revisa que el cálculo se corte en los estados terminales. Ver E10.
8	Claude Code	Proponer y ejecutar al menos tres iteraciones justificadas, incluida la comparación con presupuesto de 150 mm.	Cuatro iteraciones: dos adoptadas, Q inicial optimista con α decreciente y quitar el pronóstico en Q-learning, y dos descartadas.	Satisfactorio. Cuando la hipótesis del presupuesto resultó falsa, no forzó el resultado: probó variantes de la regla para medir el margen y encontró la causa en el simulador. Las mejoras menores que el ruido no se adoptaron.
9	Claude Code	Evaluar todas las estrategias en los años de prueba y generar las figuras finales para el documento y la presentación.	Tablas de resultados en 8 segundos, sin reentrenar, y cuatro figuras a 300 dpi.	Parcial. Las tablas fueron correctas, pero dos figuras salieron con defectos visibles que se corrigieron al revisarlas. Ver E7.

N.º	Herramienta	Prompt	Resultado	Evaluación
10	Claude Code	Generar la animación de una temporada, documentar los agentes y comprobar que todo el proyecto se reproduce desde cero.	Animación del agente frente a la regla por umbral, documentación completa y pipeline de 10.4 minutos con resultados idénticos.	Satisfactorio. La temporada de la animación muestra al agente peor que el umbral y se dejó así, documentada, en lugar de buscar una más favorable. Ver E8.

Tabla 7

Respuestas con errores y su corrección

Ref.	Problema	Por qué creemos que ocurrió	Corrección
E1 (2)	El prompt ordenaba hacer un commit al final de cada fase.	Asumió un flujo de desarrollo estándar sin considerar cómo queríamos manejar el repositorio.	Los commits los hace cada integrante después de revisar el código.
E2 (3)	La descarga de datos climáticos falló por tiempo de espera.	Pidió 26 años de datos diarios en una sola petición a la API.	La herramienta cambió la descarga a tramos de 3 años con reintentos.
E3 (3)	El código tenía demasiados comentarios.	El prompt pedía comentarios orientados a estudiantes y la IA comentó casi cada línea.	En el prompt 5 se pidió dejar solo docstrings breves.
E4 (3)	El .gitignore excluía los modelos entrenados.	Aplicó la convención de no versionar modelos sin considerar que la evaluación final usa modelos entrenados en otra computadora.	Se eliminaron esas líneas antes del primer commit.
E5 (5)	Una prueba falló al agregar el presupuesto.	La prueba armaba a mano los datos del entorno y no incluía el nuevo campo.	La herramienta actualizó la prueba; pasaron las 34.
E6 (6)	Q-learning regaba casi tres veces más que la regla por umbral.	La IA siguió los hiperparámetros pedidos sin cuestionarlos: con Q inicial en 0 y α constante, el agente nunca exploró el suelo seco y su política quedó ruidosa.	Q inicial optimista y α decreciente, con lo que el riego bajó a 216 mm.
E7 (9)	Barras de error con valores negativos y puntos de la gráfica de dispersión fuera de rango.	Aplicó la desviación estándar de forma simétrica sin considerar que las métricas no pueden ser negativas, y usó un desplazamiento aleatorio de tamaño fijo.	Se recortó el error inferior en 0 y el desplazamiento se hizo proporcional a la escala.
E8 (10)	La primera corrida del pipeline completo falló.	Se ejecutó desde la carpeta de resultados y no encontró el entorno virtual por usar rutas relativas.	Se volvió a ejecutar con rutas absolutas y se comprobó que los resultados fueran idénticos.
E9 (6)	El prompt pedía leer `docs/EXPLICACION_CODIGO.md` y `docs/ITERACIONES.md` antes de empezar, pero no estaban en el repositorio.	El prompt se escribió suponiendo que la documentación de la base ya estaba subida.	La herramienta lo reportó y creó ambos documentos desde cero a partir del código.
E10 (7)	En el registro afirmó que usar un solo hilo de PyTorch hacía el entrenamiento más rápido, sin haberlo medido.	Presentó una explicación plausible como si fuera un resultado verificado.	Se detectó al revisar y se cambió por la razón real: entrenar las tres semillas en paralelo sin que compitan por los núcleos.

Los errores tuvieron tres causas. La más común fue que la herramienta asumió información que no le dimos o que no conocía, como la forma en que el equipo maneja el repositorio (E1, E4,

E9). La segunda fue seguir lo que se le pidió aunque no fuera la mejor opción, como los hiperparámetros iniciales de Q-learning (E6). La tercera, que ocurrió una vez, fue presentar como verificado algo que no se había medido (E10). Todos los detectamos al revisar resultados, pruebas o figuras.

La IA hizo mucho más rápida la implementación de la base y de los agentes. Donde más ayudó, sin embargo, fue en el análisis: propuso el presupuesto de agua y, cuando la hipótesis falló, ayudó a medir el margen real con variantes de la regla y a encontrar la causa en el simulador. Las decisiones fueron nuestras: el tema, el alcance, qué iteraciones adoptar y cómo reportar los resultados.

6. Resultados

La Tabla 8 presenta la evaluación final en el conjunto de prueba: 12 temporadas entre 2022 y 2025, con las semillas de evaluación 0, 1 y 2. Cada estrategia de referencia suma 36 episodios y cada agente 108, porque se evalúan tres modelos entrenados con semillas distintas. Se reporta la media y la desviación estándar entre episodios. La productividad es el rendimiento relativo por cada 100 mm de riego y no está definida cuando no se riega.

Tabla 8
Resultados en el conjunto de prueba, 2022-2025 (en negrita, la mayor recompensa de cada escenario)

Escenario	Estrategia	Agua (mm)	Días con estrés	Rend. relativo	Productividad del agua	Recompensa
Sin presupuesto	Sin riego	0 ± 0	48.2 ± 22.9	0.625 ± 0.278	n/a	99.3 ± 22.3
Sin presupuesto	Calendario	750 ± 0	0.0 ± 0.0	1.000 ± 0.000	0.13 ± 0.00	121.0 ± 3.2
Sin presupuesto	Umbral	187 ± 114	4.7 ± 2.8	0.997 ± 0.003	0.65 ± 0.35	131.3 ± 1.4
Sin presupuesto	Q-learning	232 ± 107	0.0 ± 0.0	1.000 ± 0.000	0.65 ± 0.79	130.9 ± 1.5
Sin presupuesto	DQN	193 ± 113	1.3 ± 2.4	0.999 ± 0.002	0.75 ± 0.81	**131.4 ± 1.5**
150 mm	Sin riego	0 ± 0	48.2 ± 22.9	0.625 ± 0.278	n/a	99.3 ± 22.3
150 mm	Calendario	150 ± 0	47.5 ± 22.2	0.634 ± 0.267	0.42 ± 0.18	97.1 ± 21.3
150 mm	Umbral	121 ± 45	18.7 ± 20.0	0.884 ± 0.157	0.72 ± 0.29	**121.3 ± 13.7**
150 mm	Q-learning	114 ± 44	32.5 ± 20.4	0.821 ± 0.141	0.96 ± 0.68	116.0 ± 12.1
150 mm	DQN	132 ± 36	25.7 ± 26.1	0.870 ± 0.162	0.71 ± 0.36	120.3 ± 13.4

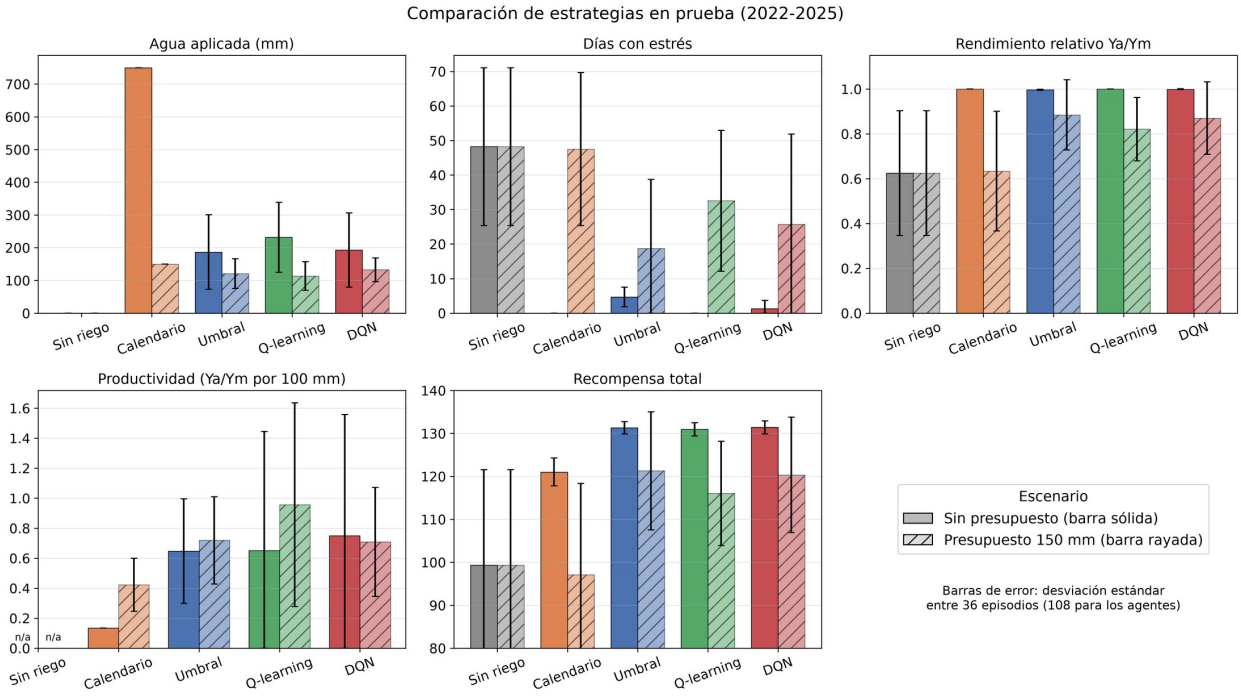


Figura 4. Comparación de las cinco estrategias en prueba. Las barras sólidas corresponden al escenario sin presupuesto y las rayadas al presupuesto de 150 mm.

Sin presupuesto, los dos agentes llegan al nivel de la regla por umbral. DQN obtiene la mayor recompensa (131.4 contra 131.3), con 1.3 días de estrés por temporada frente a 4.7 del umbral y casi la misma agua (193 contra 187 mm). La diferencia de recompensa es menor que la que hay entre semillas (de 131.3 a 131.6), así que la tomamos como un empate. Q-learning llega a un rendimiento de 1.000 pero gasta 45 mm más por temporada. Las otras dos estrategias muestran por qué la recompensa castiga los extremos: sin riego se pierde el 37 % del rendimiento, y el calendario gasta 750 mm para lograr lo mismo que el umbral con 187.

Con presupuesto de 150 mm, la regla por umbral es la mejor estrategia (121.3), seguida de DQN (120.3) y Q-learning (116.0). La Tabla 9 separa los resultados por época de siembra.

Tabla 9

Rendimiento relativo / recompensa media por época de siembra en prueba

Escenario	Estrategia	Primera	Postrera	Época seca
Sin presupuesto	Umbral	0.998 / 130.8	0.998 / 131.5	0.993 / 131.7
Sin presupuesto	Q-learning	1.000 / 130.0	1.000 / 131.0	1.000 / 131.8
Sin presupuesto	DQN	0.999 / 130.7	1.000 / 131.6	0.999 / 132.0
150 mm	Umbral	0.982 / 129.5	0.998 / 131.5	0.672 / 102.8
150 mm	Q-learning	0.893 / 122.6	0.911 / 123.3	0.659 / 102.2
150 mm	DQN	0.968 / 128.2	0.991 / 130.5	0.651 / 102.3

En la época seca, el umbral y los dos agentes gastan los 150 mm y terminan con la misma recompensa (entre 102 y 103). El agua no alcanza y no hay una forma mejor de repartirla. La diferencia aparece en primera y postrera, donde el presupuesto casi nunca se agota. Ahí la regla riega solo cuando hace falta, mientras que los agentes, entrenados para cuidar el agua, ahorran de más. DQN pierde poco (rendimiento entre 0.97 y 0.99) y Q-learning bastante más (entre 0.89 y 0.91). La tabla de Q-learning solo tiene tres niveles de agua restante y no conoce el día exacto, así que no puede distinguir entre quedarse con poca agua porque ya regó y quedarse con poca agua cuando faltan 100 días.

La Figura 5 muestra la temporada más seca del conjunto de prueba, con 42 mm de lluvia. DQN aprendió a repartir el agua de otra manera que el umbral: en lugar de reponer 30 o 40 mm cada vez que se cruza RAW, aplica dosis de 10 mm entre los días 50 y 93. Las dos estrategias terminan con un rendimiento parecido (0.68 y 0.69), algo esperable en un modelo que solo toma en cuenta el total de la temporada.

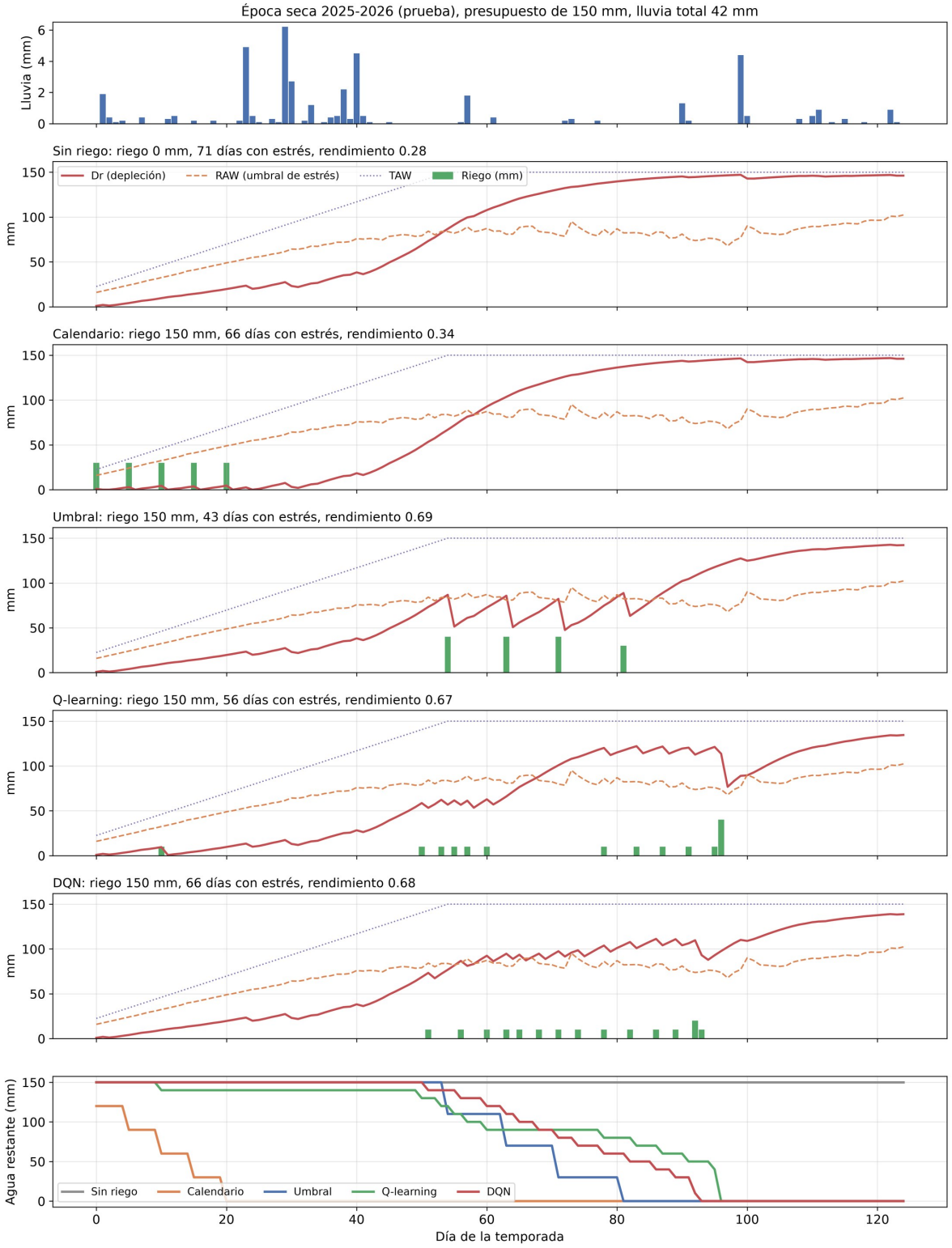


Figura 5. Época seca 2025-2026 con presupuesto de 150 mm: depleción, RAW, lluvia, riegos y agua restante de cada estrategia.

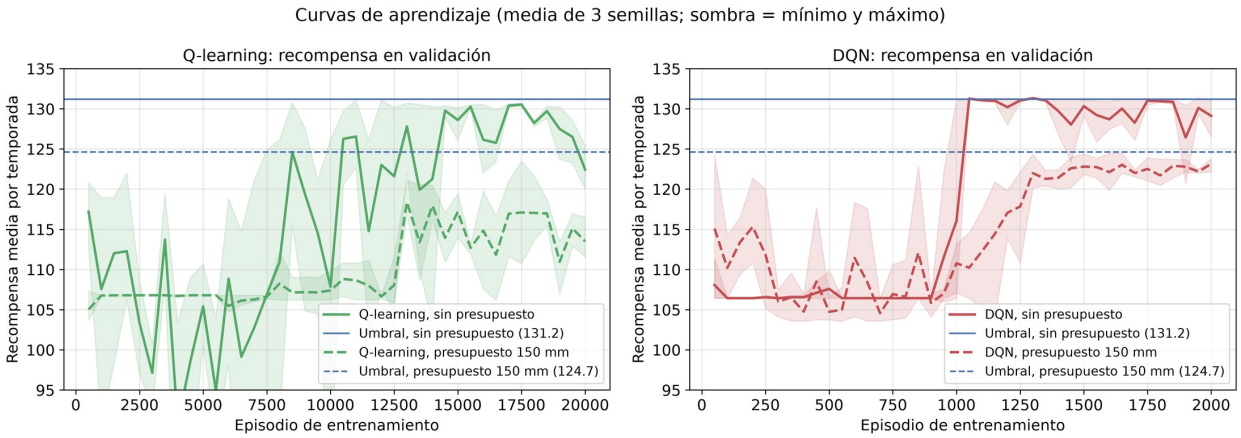


Figura 6. Curvas de aprendizaje en validación (media de tres semillas; la sombra marca el mínimo y el máximo). Las líneas azules son la regla por umbral en cada escenario.

Las curvas de aprendizaje de la Figura 6 muestran cómo aprende cada método. DQN se queda en la recompensa de no regar mientras explora y sube al nivel del umbral cuando ε se acerca a su valor final, cerca del episodio 1 000. Q-learning necesita unos 15 000 episodios y su curva tiene más ruido, porque cada celda de la tabla se estima por separado. Con presupuesto, ninguno de los dos alcanza la línea del umbral.

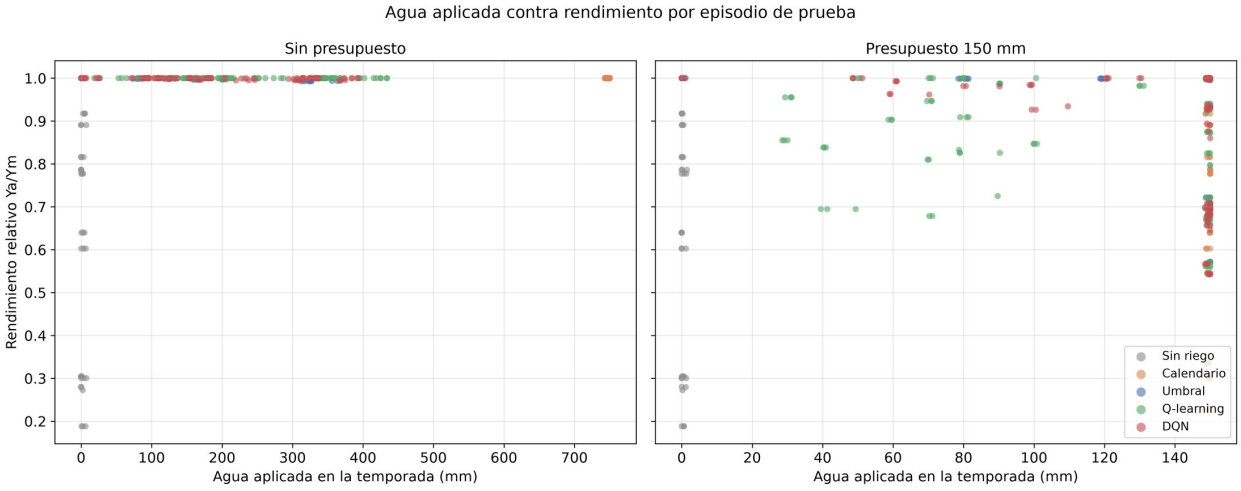


Figura 7. Agua aplicada contra rendimiento relativo en cada episodio de prueba.

7. Conclusiones

Sin presupuesto, DQN alcanzó la recompensa de la regla por umbral y tuvo menos días de estrés. Lo aprendió solo a partir de la interacción con el simulador, sin acceso a las ecuaciones del suelo.

En Q-learning, la exploración pesó más que el algoritmo. La primera versión regaba casi el triple que la regla porque nunca llegó a ver el suelo seco. Con inicialización optimista, una tasa de aprendizaje decreciente y un estado más pequeño alcanzó el nivel del umbral, y su política final se parece a la que diseñaría un agrónomo.

Con presupuesto de agua, ninguno de los agentes superó a la regla. El simulador usa un K_y único, así que un milímetro transpirado vale lo mismo en cualquier etapa y guardar agua para la floración no mejora el rendimiento. La hipótesis tenía sentido para un cultivo real, pero no para este modelo.

El pronóstico de lluvia a 3 días, con el ruido que le pusimos, no mejoró a DQN y empeoró a Q-learning al triplicar sus estados.

La regla por umbral usa el mismo modelo FAO-56 que el simulador, y por eso es tan difícil de superar. Para que el aprendizaje por refuerzo saque ventaja, el problema tendría que incluir algo que la regla no considera, como etapas con distinta sensibilidad al estrés o pronósticos de mayor alcance.

7.1 Limitaciones y trabajo futuro

El simulador usa una sola capa de suelo con parámetros fijos y una sola localidad con datos de reanálisis. Emplea un K_c único sin separar la evaporación del suelo, calcula la escorrentía de forma lineal, supone que el riego tiene eficiencia de 100 % y usa un pronóstico sintético. El conjunto de validación tiene solo 6 temporadas, por lo que las decisiones de diseño se tomaron con datos ruidosos.

Como trabajo futuro proponemos usar un K_y por etapa, como recomienda FAO-33, con lo que guardar agua para la floración tendría sentido. También valdría la pena usar pronósticos meteorológicos reales de mayor alcance, incluir el costo de bombeo y probar los agentes en otras localidades del Corredor Seco.

8. Bibliografía

Allen, R. G., Pereira, L. S., Raes, D. y Smith, M. (1998). *Crop evapotranspiration: Guidelines for computing crop water requirements* (FAO Irrigation and Drainage Paper No. 56). FAO.

Anthropic. (2026). *Claude* [Modelo de lenguaje]. https://claude.ai

Anthropic. (2026). *Claude Code* [Software]. https://claude.com/claude-code

Doorenbos, J. y Kassam, A. H. (1979). *Yield response to water* (FAO Irrigation and Drainage Paper No. 33). FAO.

FAO. (2012). *Estudio de caracterización del Corredor Seco Centroamericano*. Organización de las Naciones Unidas para la Alimentación y la Agricultura.

Harris, C. R., Millman, K. J., van der Walt, S. J. et al. (2020). Array programming with NumPy. *Nature, 585*, 357-362.

Hersbach, H., Bell, B., Berrisford, P. et al. (2020). The ERA5 global reanalysis. *Quarterly Journal of the Royal Meteorological Society, 146*(730), 1999-2049.

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90-95.

Kingma, D. P. y Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations*.

Mnih, V., Kavukcuoglu, K., Silver, D. et al. (2015). Human-level control through deep reinforcement learning. *Nature, 518*, 529-533.

Paszke, A., Gross, S., Massa, F. et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems, 32*.

Sutton, R. S. y Barto, A. G. (2018). *Reinforcement learning: An introduction* (2.ª ed.). MIT Press.

Towers, M., Kwiatkowski, A., Terry, J. et al. (2024). Gymnasium: A standard interface for reinforcement learning environments. *arXiv:2407.17032*.

Watkins, C. J. C. H. y Dayan, P. (1992). Q-learning. *Machine Learning, 8*, 279-292.

Zippenfenig, P. (2023). *Open-Meteo.com Weather API* [Software]. Zenodo. https://doi.org/10.5281/zenodo.7970649